

Introduction

Notre application est développée en java, elle à pour but de recréer un serveur Web permettant de lire des fichiers que soit des images ou du texte et de pouvoir renvoyés en fonctions de la requête les codes erreur correspondant.

Dépendances

Pour utiliser notre application vous aurez besoin d'installer java et d'avoir plusieurs fichiers de différents type (image, text...) pour tester les fonctionnalités de notre Serveur Web.

Site de téléchargement de Java : <https://www.oracle.com/java/technologies/downloads/>

Quick Start

Avant le Lancement de notre application vous devez tout d'abord compiler les fichiers avec la commande « javac WebServerApplication.java » puis démarrer l'application avec la commande « java WebServerApplication ».

Notes

Suite à l'exécution de l'application ouvrir votre navigateur favori et rendez-vous à l'adresse suivante : <http://127.0.0.1:8080>

Une fois l'adresse ouverte dans votre navigateur ajouter à la suite de l'url « /nomDeVotreFichier.Extension » en remplaçant par le fichier que vous souhaitez consulter.

Et voilà si tout se passe bien votre fichier devrait apparaitre dans votre navigateur.

V0.1)

La V0.1 comporte 2 fichier java le premier WebServerApplication n'a rien de particulier c'est une simple classe avec une méthode main et celle-ci créer une instance du deuxième fichier WebServer avec en paramètre le port 8080 qui est un port de test pour notre application.

WebServer comporte quant à lui un constructeur et trois méthodes :

- socketserver : cette méthode créer un nouveau socket et lance le server qui écoute les requêtes du client.
- readRequest : cette méthode lit la requête du client et renvoie la plupart du temps « GET / HTTP/1.1 ».

- sendReponse : cette méthode renvoie la réponse du server au client en envoyant dans l'entête HTTP « HTTP/1.0 200 ok » ce qui signifie que le server à bien compris la requête il renvoie donc un code 200 (succès de la requête).

Les codes http : https://fr.wikipedia.org/wiki/Liste_des_codes_HTTP

Documentations socket (url raccourci) : <https://urlr.me/vLzMS>

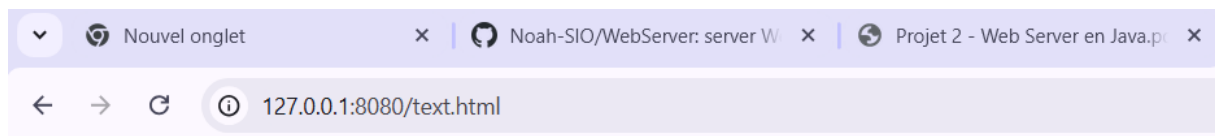
Sur votre navigateur vous devez avoir une page totalement blanche.

V0.2)

Cette version V0.2 apporte la possibilité de lire des fichiers html grâce à la modification de la méthode sendReponse, nous ajoutons alors dans l'entête HTTP le type du contenu que nous souhaitons que le server lise « Content-Type: text/html » dans ce cas c'est un fichier html.

Nous avons ainsi ajouté BufferedReader et un FileReader permettant d'ouvrir le fichier et de lire son contenu, le fichier est par la suite petit à petit ajouté dans une chaîne de caractères puis cette chaîne est envoyée au server.

En démarrant l'application et en vous rendant sur l'url du server et en ajoutant votre fichier html à la suite de l'url, vous devez normalement avoir l'affichage de votre fichier qui s'effectue.



Page de test de mon serveur web

Ceci est la page de test de mon serveur web, si vous pouvez lire ceci c'est que tout fonctionne correctement

V0.3)

Dans la version V0.3 le processus, la requête, la réponse et le contexte ont chacun leur propre classe et fichier java ce qui permet de réorganiser le code et d'avoir plus de facilité à l'utiliser.

- WebServer devient alors un fichier permettant de lancer tout le processus de l'application grâce à sa méthode run qui crée ainsi le socket et lance le server.
- HttpReponse hérite des méthodes ok et notFound permettant d'envoyer ou non un code erreur en fonction de la requête reçue par le server.
- HttpRequest permet pour l'instant de lire la requête du client.
- HttpContext permet de renvoyer des messages d'erreur et de fermer le server en cas de problème.
- HttpProcessor quant à lui permet de vérifier si la requête comprend un slash et appelle alors la réponse correspondante.

Si vous rajoutez « http://127.0.0.1:8080/**coucou** » à la fin de l'url celui-ci vous renverra une page d'erreur grâce à l'entête HTTP contenant le code erreur 404 et au contraire si vous ne mettez rien derrière l'url celui-ci vous renverra la même page blanche qu'en V0.1 signifiant que la requête fonctionne.

V0.4)

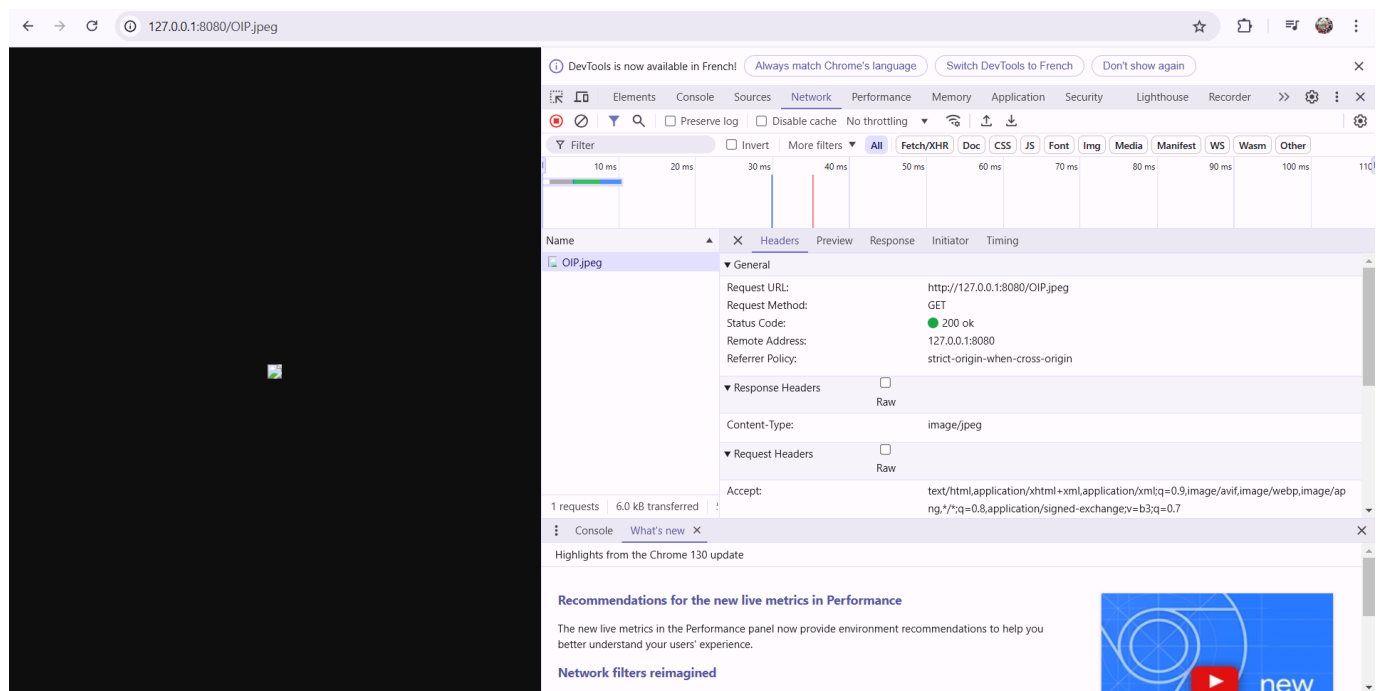
Enfin dans la version 0.4 nous avons rajouter et modifier des méthodes afin de pouvoir lire avec notre server web des fichiers de plusieurs type comme des images ou du texte, dans la classe HttpRequest nous avons modifier la méthode getUrl, elle permet maintenant de récupérer la fin de l'url de notre requête et tester si le fichier correspondant existe ou non.

Nous avons implémenté les méthode sendFile et sendContent, l'une permet d'envoyer les entêtes correspondants au type du fichier tandis que l'autre renvoie le contenu du fichier tout cela en binaires.

Pour le bon fonctionne de cette version nous avons aussi modifier la méthode process de la classe RequestProcessor qui permet alors de tester la fin de l'url afin d'appeler les méthodes avec les bons paramètres. Si la requête se termine par .html nous allons alors renvoyer en paramètre « Content-Type: text/html » ainsi que le contenu du fichier si celui-ci est présent.

Point à compléter

Vous devez savoir que notre projet fonctionne dans sa globalité mais pour l'affichage des images nous rencontrons un problème que nous pourrions régler dans une prochaine version.



Sur cette image vous pouvez constater que sur la droite notre image est reconnue en tant que jpeg avec un code 200 ce qui signifie que la requête est comprise par le server mais sur la gauche notre image ne s'affiche pas correctement et laisse place à une icône sur un fond noir.

Questions du projet Http

Question : Donnez le contenu de la requête envoyée par votre navigateur, si vous essayez d'analyser cette requête que contient-elle selon vous ? Vous devez constater que cette requête se compose 3 blocs différents, quels sont-ils ?

Ncat: Version 7.95 (<https://nmap.org/ncat>)

Ncat: Listening on 0.0.0.0:8080

Ncat: Connection from 127.0.0.1:50134.

GET / HTTP/1.1

Host: 127.0.0.1:8080

Connection: keep-alive

Cache-Control: max-age=0

sec-ch-ua: "Microsoft Edge";v="129", "Not=A?Brand";v="8", "Chromium";v="129"

sec-ch-ua-mobile: ?0

sec-ch-ua-platform: "Windows"

Upgrade-Insecure-Requests: 1

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/129.0.0.0 Safari/537.36 Edg/129.0.0.0

Accept:

text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

Sec-Fetch-Site: none

Sec-Fetch-Mode: navigate

Sec-Fetch-User: ?1

Sec-Fetch-Dest: document

Accept-Encoding: gzip, deflate, br, zstd

Accept-Language: fr,fr-FR;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6

Cette requête contient les informations d'ip du client et du server que nous avons essayé de contacter ainsi que plusieurs autres informations comme le navigateur, le système d'exploitation...

Question : Là aussi si vous analysez la réponse reçue vous devez être capable de distinguer plusieurs blocs différents, essayez de les identifier et de comprendre ce qu'ils représentent

[illegible]

Bloc option / date / cache/ information server...

Bloc html de la page.

Question 2 : Réalisez plusieurs tests, par exemple en appelant une page différente de la racine et qui ne se trouve pas sur le serveur, que constatez-vous ?

```
C:\Users\Utilisateur>telnet www.google.com/coucou 80
Connexion à www.google.com/coucou...Impossible d'ouvrir une connexion à l'hôte, sur le port 80: Échec lors de la connexion
```

Un message d'erreur apparait échec de la connexion, ce qui signifie que sur google.com il n'existe pas de page appelé coucou donc le server renvoie un message d'erreur.